

FIT5217

Natural Language Processing

Week 2 - Language Modelling

Trang Vu

(Slides and material built on the foundational work of Jurafsky & Martin; Kai-Wei Chang; Chris D. Manning; Yoav Artzi; Luke Zettlemoyer; Greg Durrett; Dan Klein; Alane Suhr; Mausam Jain, Tanmoy Chakraborty—along with our former CE, Ehsan Shareghi, who curated the majority of the content.)



Overview

1. **What is a language model?**
2. Context Length
3. The Chain Rule of probability
4. n-gram language models
5. Data sparsity issues
6. Smoothed n-grams
7. Evaluating model performance

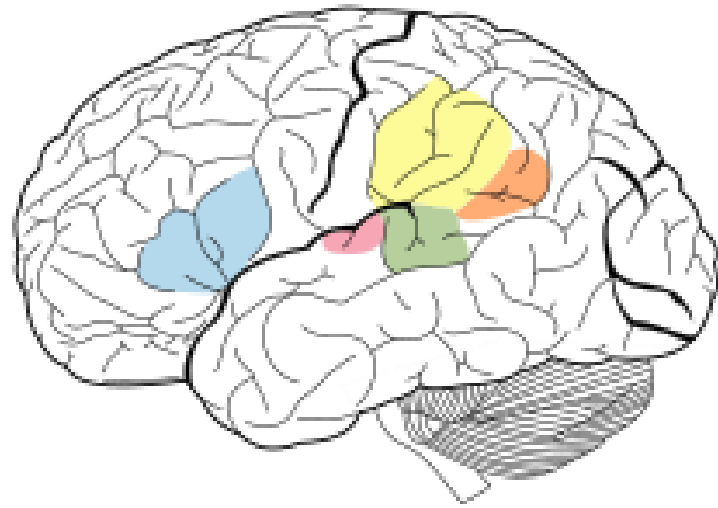
Do we need to model a language?



Do we need to model a language?

Named must be your fear before banish it you can.

It is English, but it is not a proper sentence.



Your fear must be named before you can banish it.

It is English, and it is a proper sentence.

What is a Language Model formally?

For a sequence of length N , a language model calculates its probability as $P(w_1, w_2, \dots, w_N)$, compactly written as $P(w_1^N)$. The probability indicates how likely it is for that sequence to belong to a language (e.g., training corpus domain).

$P(\text{Named must be your fear before banish it you can.})$

$<$

$P(\text{Your fear must be named before you can banish it.})$

What is a Language Model formally?

Goal: compute the probability of a sentence or sequence of words W :

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4) \text{ or } P(w_n | w_1, w_2 \dots w_{n-1})$$

An LM computes either of these:

$$P(W) \text{ or } P(w_n | w_1, w_2 \dots w_{n-1})$$



P(Named must be your fear before banish it you can.)

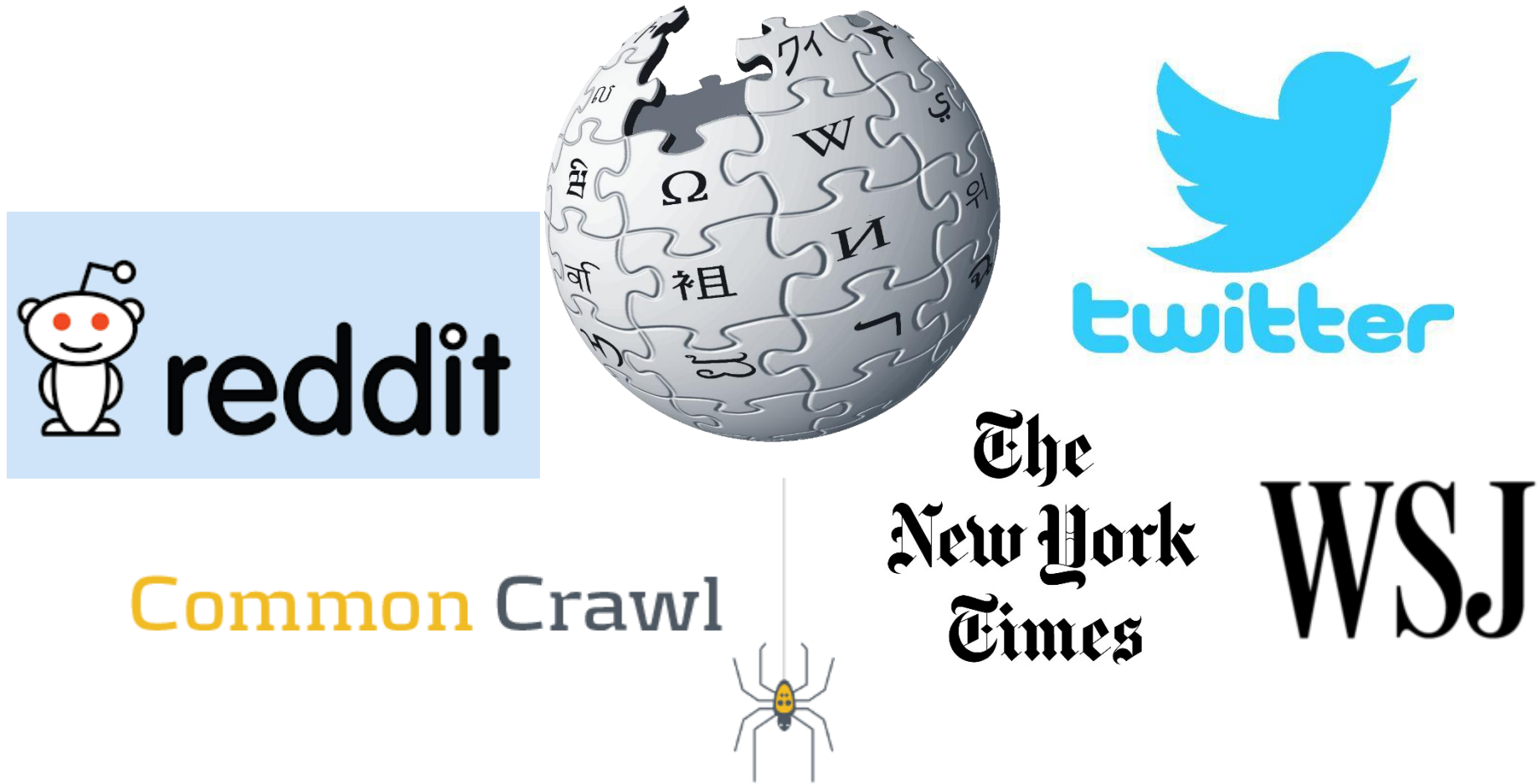
<

P(Your fear must be named before you can banish it.)

*Note: let's call these words for now, but for LLMs we instead use tokens

https://web.stanford.edu/~jurafsky/slp3/slides/lm_jan25.pdf

probabilities are calculated on a training text corpus



Why Language Modelling? Speech Recognition



$P(\text{I read the whole book}) > P(\text{I red the hole book})$

Why Language Modelling? Automatic Translation

Président de la **Chambre** des représentants

President of the **Bedroom** of Representatives
President of the **House** of Representatives

$P(\text{House} \mid \text{President of the}) > P(\text{Bedroom} \mid \text{President of the})$

Why Language Modelling? Predictive Typing



* Courtesy of Forbes.

Donald trump is a p

donald trump is a **pokemon**

donald trump is a **potato**

donald trump is a **populist**

donald trump is a **politician**

donald trump is a **pragmatist**

donald trump is a **prophet**

donald trump is a **piece of garbage**

donald trump is a **pendejo**

$P(\text{pokemon} \mid \text{Donald trump is a}) > P(\text{politician} \mid \text{Donald trump is a})$

Overview

1. What is a language model?
- 2. Context Length**
3. The Chain Rule of probability
4. n-gram language models
5. Data sparsity issues
6. Smoothed n-grams
7. Evaluating model performance

What is context?



* Courtesy of Forbes.

- Q donald trump is
- Q donald trump is **republican or democrat**
- Q donald trump is **the president of which country**
- Q donald trump is **sick**
- Q donald trump is **republican**
- Q donald trump is **what number president**
- Q donald trump is **alive**
- Q donald trump is **from which country**
- Q donald trump is **a convicted felon**
- Q donald trump is **in the epstein files**

$P(\text{Donald trump is a politician})$ vs $P(\text{politician} | \text{Donald trump is a})$

What is context?



* Courtesy of Forbes.

Donald trump is a p

donald trump is a **pokemon**

donald trump is a **potato**

donald trump is a **populist**

donald trump is a **politician**

donald trump is a **pragmatist**

donald trump is a **prophet**

donald trump is a **piece of garbage**

donald trump is a **pendejo**

$P(\text{ | Donald trump is a } p) \text{ vs } P(\text{politician | Donald trump is a } p)$

How much context is enough?

Predict the word following the sequence Donald Trump is a

Which of the following will give a more accurate prediction?

Donald Trump is a ?

Trump is a ?

is a ?

a ?

How much context is enough?

Predict the word following the sequence Donald Trump is a

Which of the following will give a more accurate prediction?

Donald Trump is a ?

Trump is a ?

is a ?

a ?



How much context is enough?

Predict the word following the sequence Donald Trump is a

Which of the following will give a more accurate prediction?

Donald Trump is a ?

Trump is a ?

is a ?

a ?

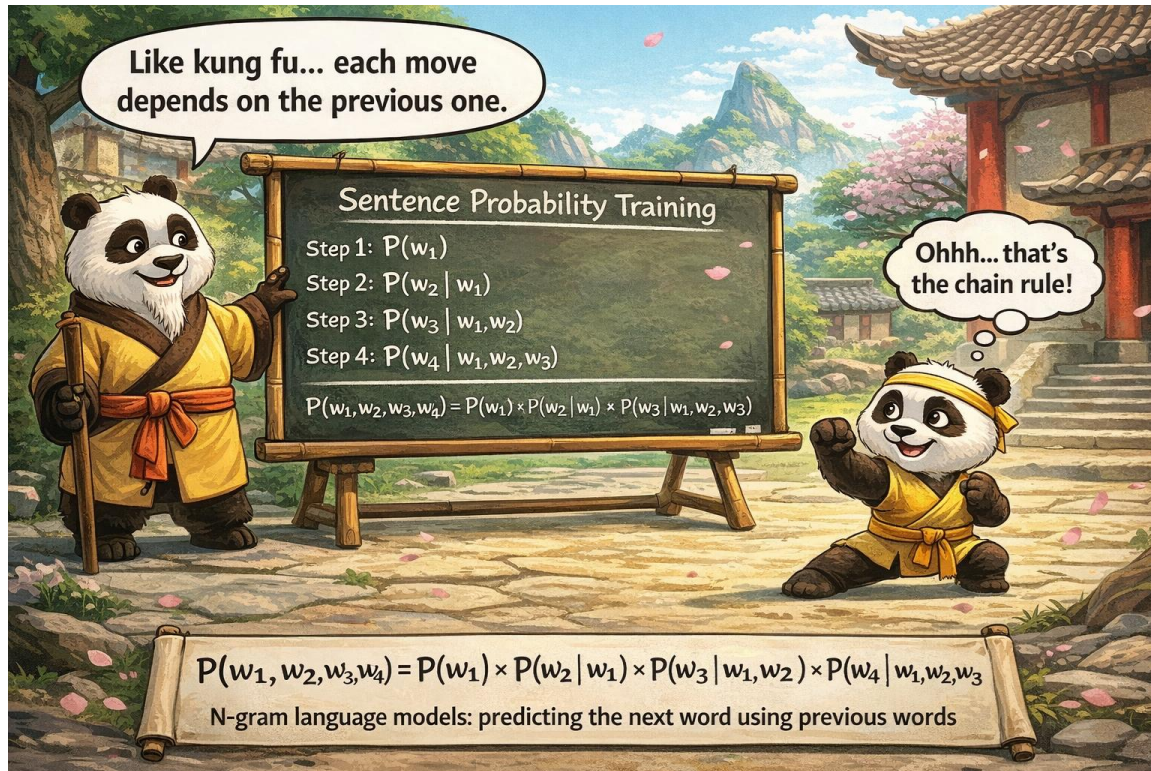


DESIRED: Ideally, we would like to consider the entire context for a more accurate prediction.

Overview

1. What is a language model?
2. Context Length
- 3. The Chain Rule of probability**
4. n-gram language models
5. Data sparsity issues
6. Smoothed n-grams
7. Evaluating model performance

The chain rule



Generated image

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

$$P(A, B) = P(A|B)P(B)$$

The chain rule

$$P(A|B) = \frac{P(A, B)}{P(B)} \qquad P(A, B) = P(A|B)P(B)$$

Let's apply it to $P(w_1^N)$ i.e., $P(W)$, the joint probability over the words/ sentence probability

$$\begin{aligned} P(w_1, w_2, \dots, w_N) &= P(w_N | w_1, \dots, w_{N-1}) P(w_1, w_2, \dots, w_{N-1}) \\ &= P(w_N | w_1, \dots, w_{N-1}) P(w_{N-1} | w_1, \dots, w_{N-2}) P(w_1, w_2, \dots, w_{N-2}) \\ &= P(w_N | w_1, \dots, w_{N-1}) P(w_{N-1} | w_1, \dots, w_{N-2}) P(w_{N-2} | w_1, \dots, w_{N-3}) P(w_1, w_2, \dots, w_{N-3}) \\ &= P(w_N | w_1, \dots, w_{N-1}) P(w_{N-1} | w_1, \dots, w_{N-2}) P(w_{N-2} | w_1, \dots, w_{N-3}) \dots P(w_2 | w_1) P(w_1) \end{aligned}$$

The chain rule

$$P(A|B) = \frac{P(A, B)}{P(B)} \qquad P(A, B) = P(A|B)P(B)$$

Let's apply it to $P(w_1^N)$

$$\begin{aligned} P(w_1, w_2, \dots, w_N) &= P(w_N | w_1, \dots, w_{N-1}) P(w_1, w_2, \dots, w_{N-1}) \\ &= P(w_N | w_1, \dots, w_{N-1}) P(w_{N-1} | w_1, \dots, w_{N-2}) P(w_1, w_2, \dots, w_{N-2}) \\ &= P(w_N | w_1, \dots, w_{N-1}) P(w_{N-1} | w_1, \dots, w_{N-2}) P(w_{N-2} | w_1, \dots, w_{N-3}) P(w_1, w_2, \dots, w_{N-3}) \\ &= P(w_N | w_1, \dots, w_{N-1}) P(w_{N-1} | w_1, \dots, w_{N-2}) P(w_{N-2} | w_1, \dots, w_{N-3}) \dots P(w_2 | w_1) P(w_1) \end{aligned}$$

We compactly write this as:

$$P(w_1^N) = \prod_{i=1}^N P(w_i | w_1^{i-1})$$

The chain rule - example

$$\begin{aligned} P(\text{Named must be your fear before banish it you can .}) = \\ P(\text{Named} | -) \times P(\text{must} | \text{Named}) \times P(\text{be} | \text{Named must}) \times \\ P(\text{your} | \text{Named must be}) \times P(\text{fear} | \text{Named must be your}) \times \\ P(\text{before} | \text{Named must be your fear}) \times \dots \times \\ P(. | \text{Named must be your fear before banish it you can}) \end{aligned}$$

Overview

1. What is a language model?
2. Context Length
3. The Chain Rule of probability
- 4. n-gram language models**
5. Data sparsity issues
6. Smoothed n-grams
7. Evaluating model performance

Computing $P(w_1^N) = \prod_{i=1}^N P(\underbrace{w_i}_w | \underbrace{w_1^{i-1}}_u)$

What does $P(w|u)$ measure?

Donald Trump is a ?

u w

$P(w|u)$: multinomial distribution over the vocabulary indicating how likely it is for each word to appear after the sequence u

<i>a</i>			<i>bird</i>			<i>conservative</i>			<i>married</i>			<i>president</i>			<i>republican</i>			<i>university</i>		<i>zyzzyva</i>
0	...	...	0	...	...	.18	...	...	.14	...	...	.32	...	.26	...	...	0	...	0	

Computing $P(w|u)$ - Maximum Likelihood Estimation

Donald Trump is a **X**

$$P(\text{**x** = president | Donald Trump is a}) = \frac{\text{count}(\text{Donald Trump is a president})}{\text{count}(\text{Donald Trump is a})}$$

The
New York
Times

Compactly written as:

$$P(w|u) = \frac{C(uw)}{C(u)}$$

Computing $P(w|u)$ – Rough Number of Parameters

Context Size = 0, Dictionary Size = 100, Number of parameters = 100 – 1 = 99

$$P(\text{The dog chased it}) = P(\text{The}) * P(\text{dog}) * P(\text{chased}) * P(\text{it})$$

Essentially need to assign a probability to each word of the vocabulary

Computing $P(w|u)$ – Rough Number of Parameters

Context Size = 0, Dictionary Size = 100, Number of parameters = $100 - 1 = 99$

$$P(\text{The dog chased it}) = P(\text{The}) * P(\text{dog}) * P(\text{chased}) * P(\text{it})$$

Essentially need to assign a probability to each word of the vocabulary

Context Size = 1, Dictionary Size = 100, Number of parameters = $100 \times (100 - 1) = 9900$

$$P(\text{The dog chased it}) = P(\text{The} | -) * P(\text{dog} | \text{The}) * P(\text{chased} | \text{dog}) * P(\text{it} | \text{chased})$$

List all dictionary words and assign a probability to each word following them

Computing $P(w|u)$ – Rough Number of Parameters

Context Size = 0, Dictionary Size = 100, Number of parameters = $100 - 1 = 99$

$$P(\text{The dog chased it}) = P(\text{The}) * P(\text{dog}) * P(\text{chased}) * P(\text{it})$$

Essentially need to assign a probability to each word of the vocabulary

Context Size = 1, Dictionary Size = 100, Number of parameters = $100 \times (100 - 1) = 9900$

$$P(\text{The dog chased it}) = P(\text{The} | -) * P(\text{dog} | \text{The}) * P(\text{chased} | \text{dog}) * P(\text{it} | \text{chased})$$

List all dictionary words and assign a probability to each word following them

Context Size = 10, Number of parameters = ??? (Homework!)

This is toy scale! Oxford dictionary size is 171,476!

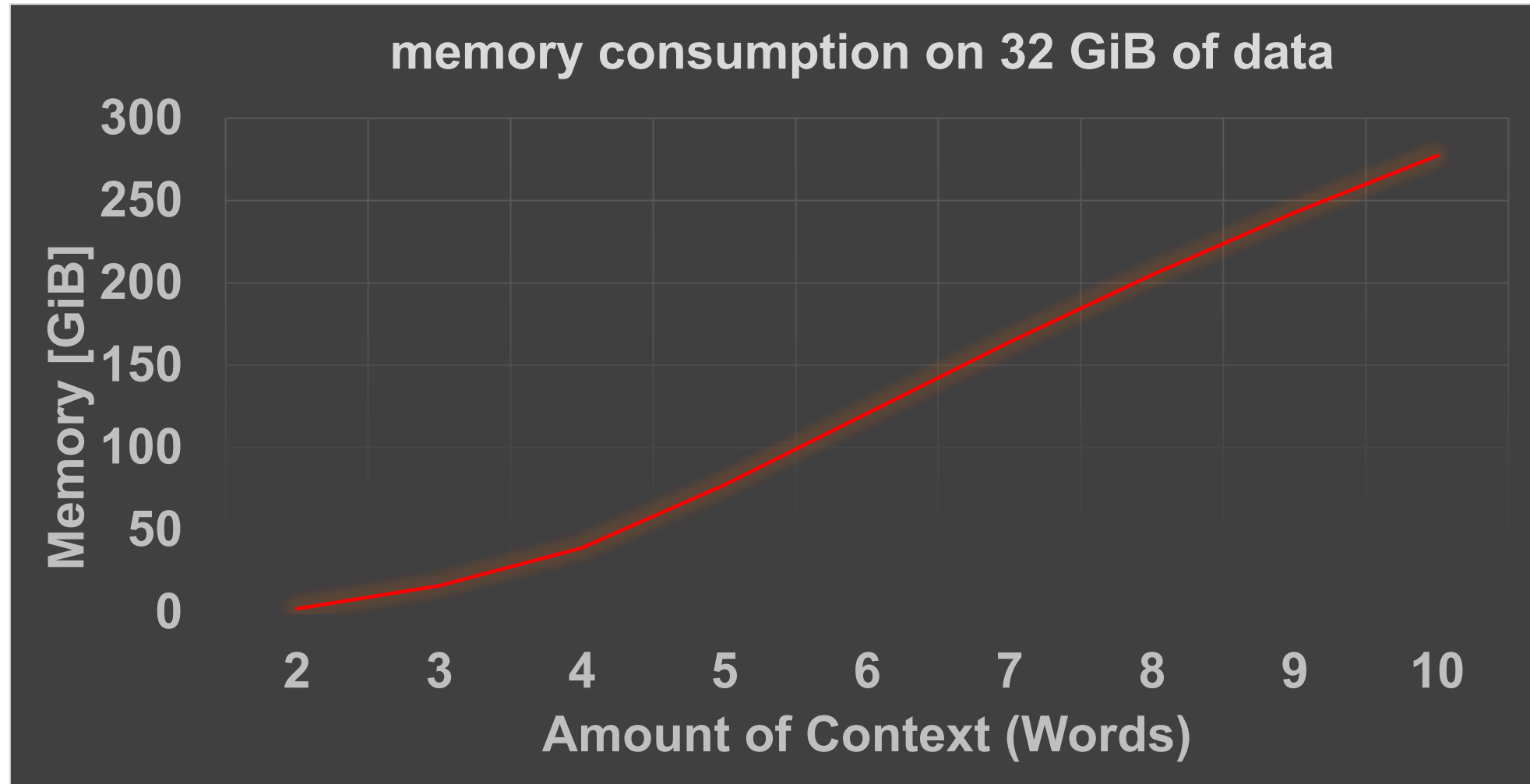
Number of parameters grows exponentially

Context Matters

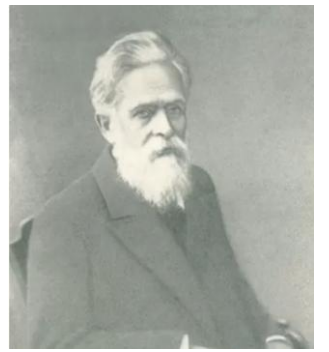
1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

Figure 3.4 Eight sentences randomly generated from four n-grams computed from Shakespeare's works. All characters were mapped to lower-case and punctuation marks were treated as words. Output is hand-corrected for capitalization to improve readability.

Context Size vs. Memory Usage for Estimation



Markov Assumption (n-gram LM)



Andrey Andreyevich **Markov**
(1856 – 1922),
Russian Mathematician

- n-gram models are probabilistic models that predict the next word in a sentence given the $n-1$ preceding words of context.
 - 1-grams (*unigrams*) predict words based on zero words of context
 - 2-grams (*bigrams*) predict words based on one word of context
 - 3-grams (*trigrams*) prediction words based on two words of context etc. etc.
- The *Markov assumption* is the assumption that future behavior only depends on recent history. In a k -th order Markov model, the next state depends only on the most recent k states.

Source: <https://www.americanscientist.org/article/first-links-in-the-markov-chain>

Markov Assumption (n-gram LM)

$$P(w_1^N) = \prod_{i=1}^N P(w_i | w_1^{i-1}) \approx \prod_{i=1}^N P(w_i | w_{i-\textcolor{red}{n}+1}^{i-1})$$

Markov Assumption (n-gram LM)

$P(\text{Named must be your fear before banish it you can .}) =$

$$\begin{aligned} &P(\text{Named} | -) * \\ &P(\text{must} | -) * \\ &P(\text{be} | -) * \\ &P(\text{your} | -) * \\ &P(\text{fear} | -) * \\ &P(\text{before} | -) * \\ &\dots * \\ &P(. | -) \end{aligned}$$

Unigram LM (n=1)

Markov Assumption (n-gram LM)

$P(\text{Named must be your fear before banish it you can .}) =$

$P(\text{Named} | -) * \\ P(\text{must} | -) * \\ P(\text{be} | -) * \\ P(\text{your} | -) * \\ P(\text{fear} | -) * \\ P(\text{before} | -) * \\ \dots * \\ P(. | -)$

OR

$P(\text{Named} | -) * \\ P(\text{must} | \text{Named}) * \\ P(\text{be} | \text{must}) * \\ P(\text{your} | \text{be}) * \\ P(\text{fear} | \text{your}) * \\ P(\text{before} | \text{fear}) * \\ \dots * \\ P(. | \text{can})$

Unigram LM (n=1)

Bigram LM (n=2)

Markov Assumption (n-gram LM)

$P(\text{Named must be your fear before banish it you can .}) =$

$P(\text{Named} | -) * \\ P(\text{must} | -) * \\ P(\text{be} | -) * \\ P(\text{your} | -) * \\ P(\text{fear} | -) * \\ P(\text{before} | -) * \\ \dots * \\ P(. | -)$

Unigram LM (n=1)

OR

$P(\text{Named} | -) * \\ P(\text{must} | \text{Named}) * \\ P(\text{be} | \text{must}) * \\ P(\text{your} | \text{be}) * \\ P(\text{fear} | \text{your}) * \\ P(\text{before} | \text{fear}) * \\ \dots * \\ P(. | \text{can})$

Bigram LM (n=2)

OR

$P(\text{Named} | -) * \\ P(\text{must} | \text{Named}) * \\ P(\text{be} | \text{Named must}) * \\ P(\text{your} | \text{must be}) * \\ P(\text{fear} | \text{be your}) * \\ P(\text{before} | \text{your fear}) * \\ \dots * \\ P(. | \text{it can})$

Trigram LM (n=3)

Example

Imagine we are given the small corpus:

<s>welcome home</s>

<s>welcome back</s>

<s>welcome home</s>

<s>you are a welcome sight</s>

<s>what a welcome</s>

w_{n-1}	w_n	count $C(w_{n-1}w_n)$	probability $P(w_n w_{n-1})$
<s>	welcome	3	?
<s>	what	1	
<s>	you	1	0.20
a	welcome	2	1.00
are	a	1	1.00
back	</s>	1	1.00
home	</s>	2	1.00
sight	</s>	1	1.00
welcome	home	2	?
welcome	back	1	
welcome	sight	1	0.20
welcome	</s>	1	0.20
what	a	1	?
you	are	1	

* Slide credit goes to Nigel Collier @ Cambridge

Example

Imagine we are given the small corpus:

<s>welcome home</s>

<s>welcome back</s>

<s>welcome home</s>

<s>you are a welcome sight</s>

<s>what a welcome</s>

w_{n-1}	w_n	count $C(w_{n-1}w_n)$	probability $P(w_n w_{n-1})$
<s>	welcome	3	0.60
<s>	what	1	0.20
<s>	you	1	0.20
a	welcome	2	1.00
are	a	1	1.00
back	</s>	1	1.00
home	</s>	2	1.00
sight	</s>	1	1.00
welcome	home	2	0.40
welcome	back	1	0.20
welcome	sight	1	0.20
welcome	</s>	1	0.20
what	a	1	1.00
you	are	1	1.00

* Slide credit goes to Nigel Collier @ Cambridge

bigram

- $P(\text{chased} \mid \text{dog}) = 0.5$
- $P(\text{followed} \mid \text{dog}) = 0.5$
- $P(\text{ate} \mid \text{dog}) = 0.2$
- $P(\text{cat} \mid \text{the}) = 0.75$
- $P(\text{dog} \mid \text{the}) = 0.25$
- $P(\text{the}) = 0.6$
- $P(\text{cat}) = 0.1$
- $P(\text{dog}) = 0.05$
- $P(\text{ate}) = 0.05$
- $P(\text{chased}) = 0.15$
- $P(\text{followed}) = 0.05$
- $P(\text{the} \mid \text{chased}) = 0.9$
- ...

$P(\text{chased the dog}) > P(\text{chased the cat})$

True or False?

bigram

- $P(\text{chased} \mid \text{dog}) = 0.5$
- $P(\text{followed} \mid \text{dog}) = 0.5$
- $P(\text{ate} \mid \text{dog}) = 0.2$
- $P(\text{cat} \mid \text{the}) = 0.75$
- $P(\text{dog} \mid \text{the}) = 0.25$
- $P(\text{the}) = 0.6$
- $P(\text{cat}) = 0.1$
- $P(\text{dog}) = 0.05$
- $P(\text{ate}) = 0.05$
- $P(\text{chased}) = 0.15$
- $P(\text{followed}) = 0.05$
- $P(\text{the} \mid \text{chased}) = 0.9$
- ...

Anything wrong with the top 3 probabilities?

Overview

1. What is a language model?
2. Context Length
3. The Chain Rule of probability
4. n-gram language models
- 5. Data sparsity issues**
6. Smoothed n-grams
7. Evaluating model performance

Let's revisit our previous example

Donald Trump is a **X**

$$P(\text{**x** = president} \mid \text{Donald Trump is a}) = \frac{\text{count}(\text{Donald Trump is a president})}{\text{count}(\text{Donald Trump is a})}$$

What if we face the following situation:

count(Donald Trump is a president) = 0
count(Trump is a president) = 100

Does it mean $P(\text{**x** = president} \mid \text{Donald Trump is a})$ should be 0?

Data Sparsity

Intuitively speaking, as the length of a sequence grows it becomes less likely to find an exact match in a given text corpus!

Related Issue – Out of Vocabulary (OOV) words

What if we were given the sequence “Meet the husband of Princess Hammock” but the word “Hammock” did not occur in our training data?



* Courtesy of Friends TV Series

Does it mean $P(\text{Meet the husband of Princess Hammock})$ should be 0?

Solution

Maximum Likelihood Estimation (MLE) fails at dealing with data sparsity scenarios as it solely estimates the probabilities based on the training data.

What strategies exist to help with this?

- Ensure *the training data* adequately represents the final application
- Smoothing (Interpolation, Backoff)

Overview

1. What is a language model?
2. Context Length
3. The Chain Rule of probability
4. n-gram language models
5. Data sparsity issues
- 6. Smoothed n-grams**
7. Evaluating model performance

Smoothed n-grams

- How do we deal with n-grams which we have never seen in training?
- Zero counts give zero probability estimates for both words and sentences.
- But some zero count n-grams are valid sequences.
- Smoothing is a class of techniques that aims to address this problem.
- Intuition is to reassign probability mass from seen to unseen events whilst maintaining a joint distribution that sums to 1.

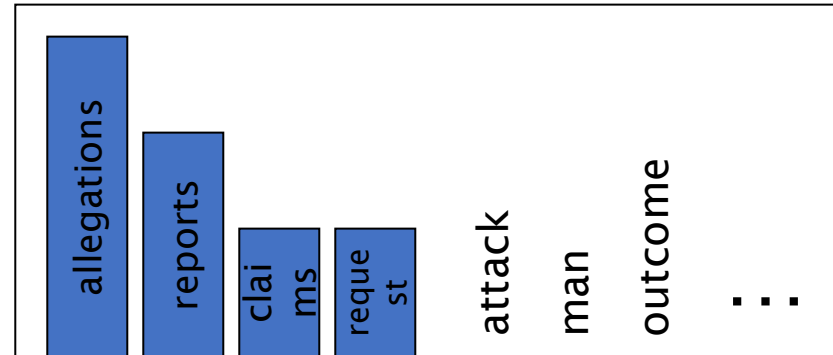
Smoothed n-grams

- When we have sparse statistics:

$P(w \mid u)$

$w \in \left\{ \begin{array}{l} 3 \text{ allegations} \\ 2 \text{ reports} \\ 1 \text{ claims} \\ 1 \text{ request} \end{array} \right.$

7 total

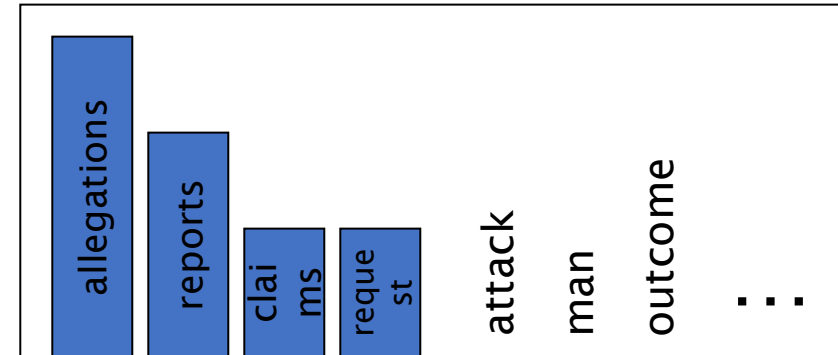


* Slides credit goes to Dan Klein/Dan Jurafsky. Data from the Wall Street Journal corpus.

Smoothed n-grams

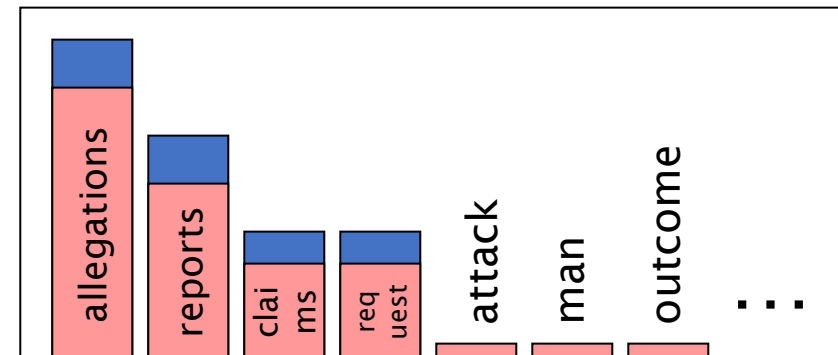
- When we have sparse statistics:

$P(w | u)$
 $w \in \left\{ \begin{array}{l} 3 \text{ allegations} \\ 2 \text{ reports} \\ 1 \text{ claims} \\ 1 \text{ request} \end{array} \right.$
 7 total



- Steal probability mass to generalize better

$P(w | u)$
 $w \in \left\{ \begin{array}{l} 2.5 \text{ allegations} \\ 1.5 \text{ reports} \\ 0.5 \text{ claims} \\ 0.5 \text{ request} \\ 2 \text{ other} \end{array} \right.$
 7 total



* Slides based on to Dan Klein/Dan Jurafsky.

Add-1 Smoothing

Recall our Maximum Likelihood Estimate:

$$P_{MLE}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Add-1 smoothing simply adds one to each count before normalization:

$$P_{Add-1}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

where V is the vocabulary size.

Question: Why do we need V in the denominator?

Example of Add-1 smoothing for $P(w | u)$

w_i				
allegations	3	3/7	4	4/13
reports	2	2/7	3	3/13
claims	1	1/7	2	2/13
request	1	1/7	2	2/13
outcome	0	0	1	1/13
fact	0	0	1	1/13
Total	7	7/7	13	13/13

unsmoothed

Add-1 smoothing

← We've reduced our likelihood by 28.2%

More data = less Add-1 smoothing

w_i				
allegations	300	300/700	301	301/706
reports	200	200/700	201	201/706
claims	100	100/700	101	101/706
request	100	100/700	101	101/706
outcome	0	0	1	1/706
fact	0	0	1	1/706
Total	700	700/700	706	706/706

unsmoothed Add-1 smoothing

← We've reduced our likelihood by 0.5%.

Larger Vocabulary = more Add-1 smoothing

- But suppose the size of our vocabulary was 2000 instead of 6:

w_i				
allegations	300	300/700	301	301/2700
reports	200	200/700	201	201/2700
claims	100	100/700	101	101/2700
request	100	100/700	101	101/2700
outcome	0	0	1	1/2700
fact	0	0	1	1/2700
John	0	0	1	1/2700
...	0	0	1993	1993/2700
Total	700	700/700	2700	2700/2700

unsmoothed
 Add-1 smoothing

Big change: We've reduced our likelihood by 73.9%

Add-1 smoothing thinks we are extremely likely to see an unknown event. Is having a big dictionary a good reason?

Add-k Smoothing

- One alternative to add-1 smoothing is to move a bit less of the probability mass from the seen to unseen events. So, instead of adding one, we add a fractional count k (i.e., $k=0.5, 0.2, 0.01$). This improves things only a little bit.

$$P_{Add-k}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + k}{C(w_{i-1}) + kV}$$

K is typically chosen on a validation or development set.

n-gram smoothing

- A large dictionary makes an unknown event too likely.
- Add-1 smoothing gives away too much probability mass.
- Is there a better way? Yes. Interpolation and backoff.

Good reference if you want to know about other smoothing techniques



DIGITAL ACCESS TO
SCHOLARSHIP AT HARVARD
DASH.HARVARD.EDU



HARVARD LIBRARY
Office for Scholarly Communication

An Empirical Study of Smoothing Techniques for Language Modeling

Citation

Chen, Stanley F. and Joshua Goodman. 1998. An Empirical Study of Smoothing Techniques for Language Modeling. Harvard Computer Science Group Technical Report TR-10-98.

Let's revisit our previous example

$$P(\textcolor{red}{x} = \text{president} \mid \text{Donald Trump is a}) = \frac{\text{count}(\text{Donald Trump is a president})}{\text{count}(\text{Donald Trump is a})}$$

$$\begin{aligned}\text{count}(\text{Donald Trump is a president}) &= 0 \\ \text{count}(\text{Trump is a president}) &= 100\end{aligned}$$

Intuition:

$$P(\text{president} \mid \text{Donald Trump is a}) \approx P(\text{president} \mid \text{Trump is a})$$

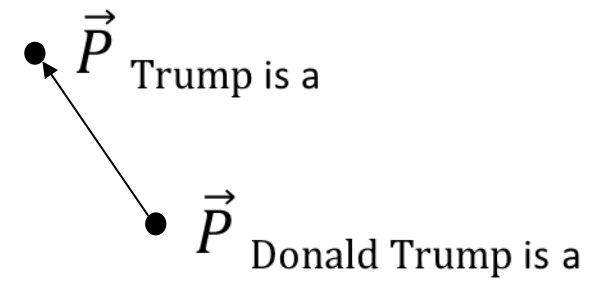
Basic Backoff Solution ($\gamma \in [0, 1]$):

$$\gamma P(\text{president} \mid \text{Donald Trump is a}) + (1 - \gamma) P(\text{president} \mid \text{Trump is a})$$

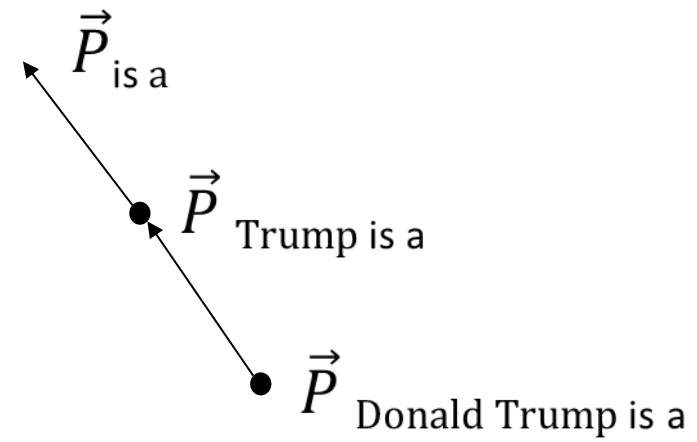
Recursion

- $\vec{P}$ Donald Trump is a

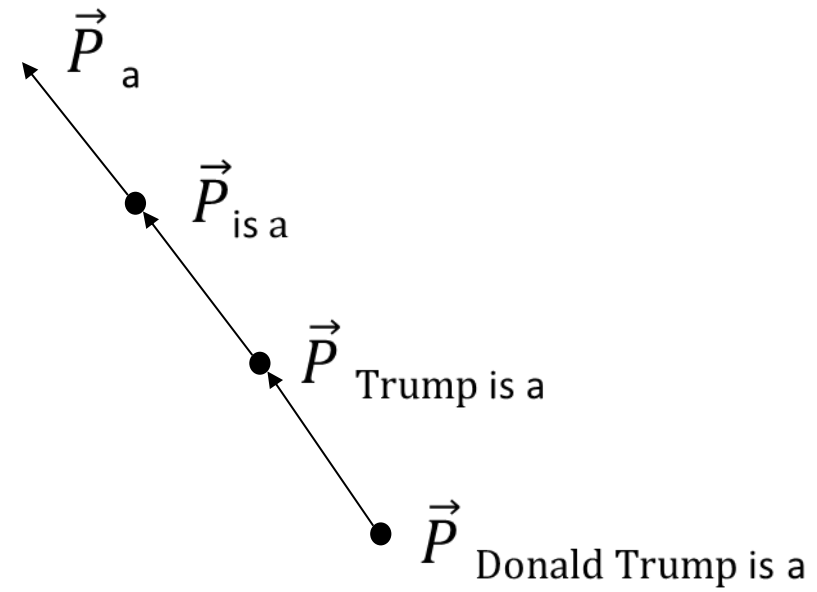
Recursion



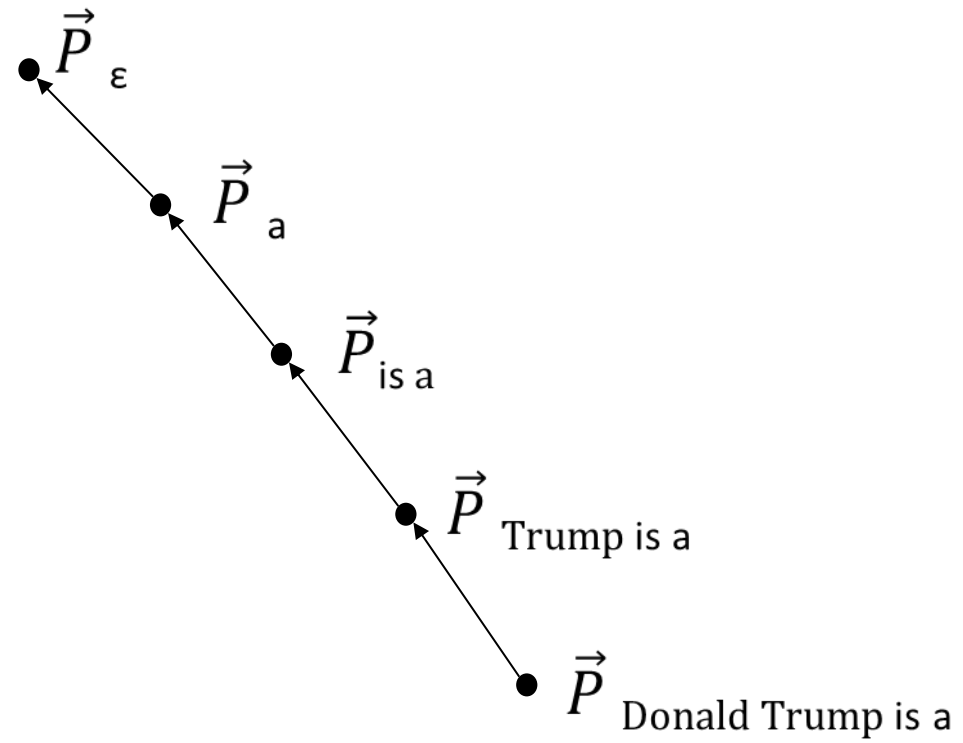
Recursion



Recursion

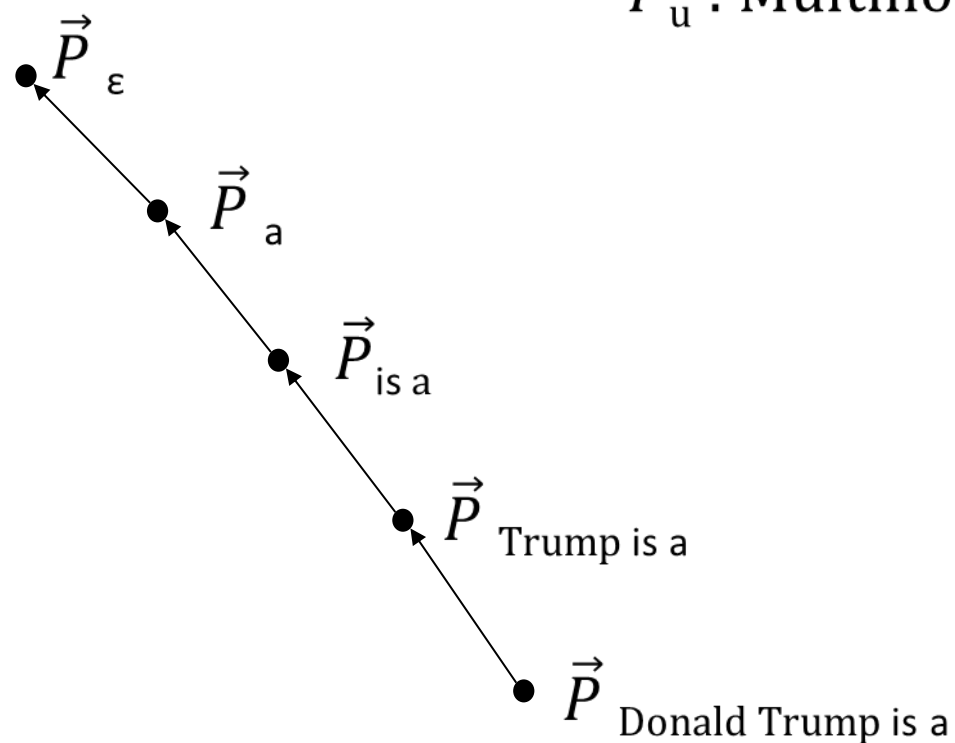


Recursion



Recursion

$\vec{P}_\epsilon$: Uniform over vocab
 $\vec{P}_u$: Multinomial over vocab



Modified Kneser-Ney Smoothing

3-gram

This is an interpolation smoothing, and you can see the **recursive** form of it.

$$P(w|ux) = \frac{c(uxw) - D_3(c(uxw))}{c(ux)} + \frac{\gamma(ux\bullet)}{c(ux)} P(w|x)$$

$$P(w|x) = \frac{N_{1+}(\bullet xw) - D_2(N_{1+}(\bullet xw))}{N_{1+}(\bullet x\bullet)} + \frac{\gamma(x\bullet)}{N_{1+}(\bullet x\bullet)} P(w| -)$$

$$P(w|-) = \frac{N_{1+}(\bullet w) - D_1(N_{1+}(\bullet w))}{N_{1+}(\bullet\bullet)} + \frac{\gamma(-\bullet)}{N_{1+}(\bullet\bullet)} * \frac{1}{|VOCAB|}$$

Not Examinable

Stupid backoff

We introduce a similar but simpler scheme, named *Stupid Backoff*¹, that does not generate normalized probabilities. The main difference is that we don't apply any discounting and instead directly use the relative frequencies (S is used instead of P to emphasize that these are not probabilities but scores):

$$S(w_i | w_{i-k+1}^{i-1}) = \begin{cases} \frac{f(w_{i-k+1}^i)}{f(w_{i-k+1}^{i-1})} & \text{if } f(w_{i-k+1}^i) > 0 \\ \alpha S(w_i | w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases} \quad (5)$$

¹The name originated at a time when we thought that such a simple scheme cannot possibly be good. Our view of the scheme changed, but the name stuck.

Large Language Models in Machine Translation

Thorsten Brants Ashok C. Popat Peng Xu Franz J. Och Jeffrey Dean

Google, Inc.

1600 Amphitheatre Parkway

Mountain View, CA 94303, USA

{brants, popat, xp, och, jeff}@google.com

Summary of Smoothing, Interpolation, and Backoff

Smoothing: Pretend you saw every n-gram one (or k) times more than you did

- A blunt instrument (replacing a lot of zeros) but sometimes useful

Backoff: If you haven't seen the trigram, use the (weighted) bigram probability instead

- Weighting is messy; "stupid" backoff works fine at web-scale

Interpolation: (weighted) mix of trigram, bigram, unigram

- Usually the best! We also use interpolation to combine multiple LLMs

Out Of Vocabulary (OOV) – a common trick

One way to handle OOV (unknown words) is to set a frequency threshold (i.e., threshold = 3) and replace all the words in the training data below this frequency threshold with a special token <UNK> at training time. This solution shares the probability mass of all the rare words in the training corpus and accumulates them under the <UNK> token. Then at test time we replace all unseen words with <UNK>. This avoids assigning zero probabilities to sequences with unseen words.

Overview

1. What is a language model?
2. Context Length
3. The Chain Rule of probability
4. n-gram language models
5. Data sparsity issues
6. Smoothed n-grams
7. **Evaluating model performance**

Evaluating a language model performance

- Evaluating model performance is a key issue in NLP.
- There are a variety of methods to evaluate NLP models. Broadly divided into:
 - *Extrinsic evaluation*: put each model into a task (e.g. MT, spelling correction), test on real-world data.
 - Realistic (best for comparing two or more models), expensive
 - *Intrinsic evaluation*: application-independent evaluations that often correlate with improvements in applications.
 - Less realistic, cheaper

Perplexity

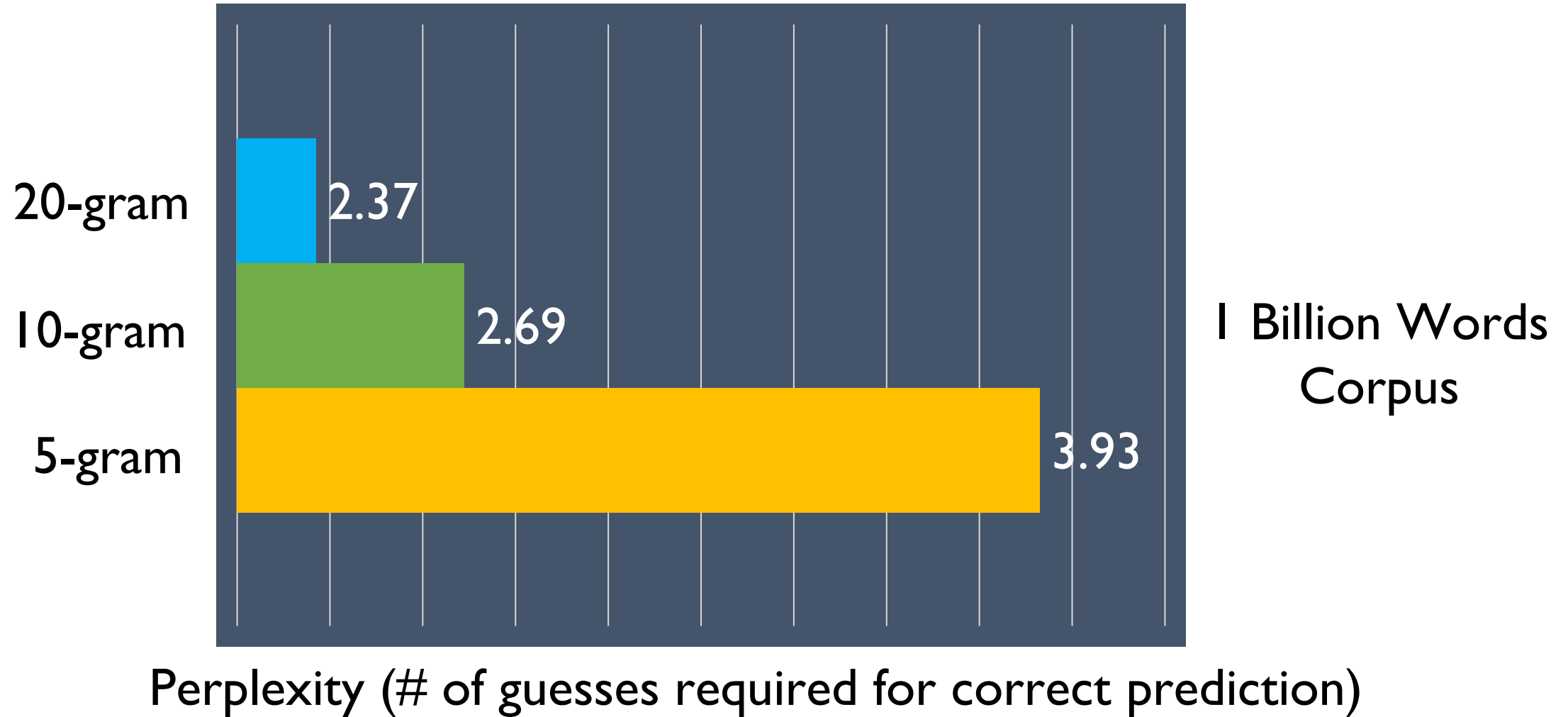
- Perplexity is the most common intrinsic evaluation metric for n-gram models.
- A measure of how well the language model ‘predicts’ the test data.
- The higher the prediction probabilities (the lower the perplexity will be), the better the model.

$$PPLX(W) = \sqrt[N]{\frac{1}{P(w_1 \dots w_N)}} = P(w_1 \dots w_N)^{-\frac{1}{N}}$$

- Can think of perplexity as the average number of guesses the model makes to correctly predict the next word.
- Lower perplexity is better.

PPLX is sometimes denoted as $2^{-\frac{1}{N} \log_2 P(w_1 \dots w_N)}$
See Sec 3.8 of Jurafsky & Martin Chapter 3.

Evaluating a language model performance



Challenges with n-gram models

1. N-grams can't handle **long-distance dependencies**:

“**The soups** that I made from that new cookbook I bought yesterday **were** amazingly delicious.”

2. N-grams don't do well at **modeling new sequences** (even if they have similar meanings to sequences they've seen)

Ex: suppose "I drove the **car**" appears 1000 times vs appears first time: "I drove the **automobile**"
. What probability will be assigned to it??

3. **Data Sparsity**, Smoothing redistributes probability mass to unseen n-grams, but blindly.

It cannot recognize that "the cat sat" and "the dog sat" are semantically related, so it treats every unseen n-gram as equally plausible regardless of meaning.

The solution: Large language models

- can handle much longer contexts
- because they use neural embedding spaces, can model meaning better

Popular n-gram LM toolkits

KenLM <https://github.com/kpu/kenlm>

SRILM <http://www.speech.sri.com/projects/srilm>

```
\1-grams:
p apple      γ
p strong     γ
...
\2-grams:
p Force is   γ
p jump over  γ
...
```

Run over the data, precompute the probabilities and coefficients of (modified) kneser-ney smoothing for the given n-gram order. Store it on a giant ARPA file that looks like this, and then use it with loads of engineering tricks at query phase as a lookup table.

Last word

But everything is now neural models. They perform so well on language modelling, why should I use an n -gram LM any ways?

Show Some Love to Your n -grams: A Bit of Progress and Stronger n -gram Language Modeling Baselines

Ehsan Shareghi, Daniela Gerz, Ivan Vulić, Anna Korhonen
Language Technology Lab, DTAL, University of Cambridge
{es776, dsg40, iv250, alk23}@cam.ac.uk

- 50 languages covering all morphological families.
- A simple extension of Modified Kneser-Ney outperforms LSTM language model on 42 languages.
- Bayesian n -gram LM outperforms the character-aware neural model on average across all languages
- n -gram models lend themselves as natural choices for resource-lean or morphologically rich languages
- n -grams are still used in NLP pipeline: pre-processing, evaluation

Infini-gram: Scaling Unbounded n -gram Language Models to a Trillion Tokens

Jiacheng Liu[♡] Sewon Min[♡]

Luke Zettlemoyer[♡] Yejin Choi^{♡♣} Hannaneh Hajishirzi^{♡♣}

[♡]Paul G. Allen School of Computer Science & Engineering, University of Washington

[♣]Allen Institute for Artificial Intelligence liujc@cs.washington.edu

Abstract

Are n -gram language models still relevant in this era of neural large language models (LLMs)? Our answer is *yes*, and we showcase their values in both text analysis and improving neural LLMs. This was done by modernizing n -gram LMs in two aspects. First, we train them at the same data scale as neural LLMs—**5 trillion tokens**. This is the largest n -gram LM ever built. Second, existing n -gram LMs use small n which hinders their performance; we instead allow n to be arbitrarily large, by introducing a new **∞ -gram LM** with backoff. Instead of pre-computing n -gram count tables (which would be very expensive), we develop an engine named **infini-gram**—powered by suffix arrays—that can compute ∞ -gram (as well as n -gram with arbitrary n) probabilities with **millisecond-level latency**. The ∞ -gram framework and infini-gram engine enable us to conduct many novel and interesting analyses of human-written and machine-generated text: we find that the ∞ -gram LM has fairly high accuracy for next-token prediction (47%), and can complement neural LLMs to greatly reduce their perplexity. When analyzing machine-generated text, we also observe irregularities in the machine- ∞ -gram agreement level with respect to the suffix length, which indicates deficiencies in neural LLM pretraining and the positional embeddings of Transformers.



Open topic for discussion

- N-gram language models estimate the probability/frequency of a sequence based on observations in a training corpus. This makes them a powerful lens for examining the issue of [memorisation](#), data contamination and [copyright](#) in large language models (LLMs).
- LLMs have been shown to generate text containing verbatim snippets from [copyrighted works](#) present in their training data.
- If a model generates a passage that matches part of a copyrighted work, how long would the passage need to be before we consider it plagiarism or copyright violation? Is length the right criterion?
- Even if the wording is different, can reproducing the style, structure, or central idea of a work raise copyright concerns? Why or why not?
- Recent studies have shown that [benchmark datasets](#) may overlap with model training data (data contamination). How does this affect our interpretation of model performance and claims about generalisation?
- Use our Ed forum "Social" space and share your thoughts.

Feedback time!



** Please submit weekly Feedback! **

